Week 3: CNN Fundamentals

Same Data, Smarter Architecture

Live Coding Session — Week 3

Why MLPs Struggle with Images

Week 1's MLP flattened 28×28 → 784. That destroys spatial structure.

MLP (Week 1)	
Input	Flatten 28×28 → 784 numbers
Spatial info	**Destroyed ✗**
Parameters	784 × 128 = 100,352 (layer 1 alone)
Learns	Global pixel-to-class mapping
Shift invariant?	No — move the object, prediction changes
Accuracy	**~87-88% on Fashion-MNIST**

Treats image as a bag of pixels — no spatial understanding

→

CNN (Today!)	
Input	Keep as 1×28×28 (C×H×W)
Spatial info	**Preserved ✓**
Parameters	3×3×32 = 288 (layer 1) — 348× fewer
Learns	Local patterns: edges → textures → parts
Shift invariant?	Yes — same filter slides across the image
Accuracy	**~90-92% on Fashion-MNIST**

Understands spatial structure — learns hierarchical features

Today's goal: Replace the MLP with a CNN on the exact same Fashion-MNIST dataset. Same training loop, better architecture, higher accuracy.

The Convolution Operation

A small filter slides across the image, detecting patterns everywhere

Input (5×5)

1	0	1	0	1
0	1	0	1	0
1	0	1	0	1
0	1	0	1	0
1	0	1	0	1

*

Kernel (3×3)

1	0	-1
1	0	-1
1	0	-1

Vertical edge detector

=

Feature Map

0	2	0
-2	0	2
0	-2	0

Strong response = edge found!

How it works

Slide the kernel across the image. At each position, element-wise multiply and sum.
Same kernel everywhere = weight sharing = parameter efficient.

Key Insight

Layer 1: Edges, gradients

Layer 2: Textures, corners

Layer 3: Parts, patterns

Deeper layers: Objects, scenes

Simple → complex, automatically

`nn.Conv2d(in_channels, out_channels, kernel_size)` — that's it. PyTorch handles the sliding and summation.

Pooling: Downsample and Generalize

After convolution, pool to reduce size and keep the strongest signals

Input (4×4)

1	3	2	1
4	6	5	2
7	8	3	1
2	4	1	0

→

MaxPool
2×2, s=2

Output (2×2)

6	5
8	3

Max Pooling — What and Why

Takes the maximum value from each 2×2 window.

Slides with stride 2 → halves spatial dimensions.

Why it matters:

1. Reduces computation (4× fewer values per layer)
2. Translation invariance — small shifts don't change the max
3. Keeps the strongest activations (most important features)

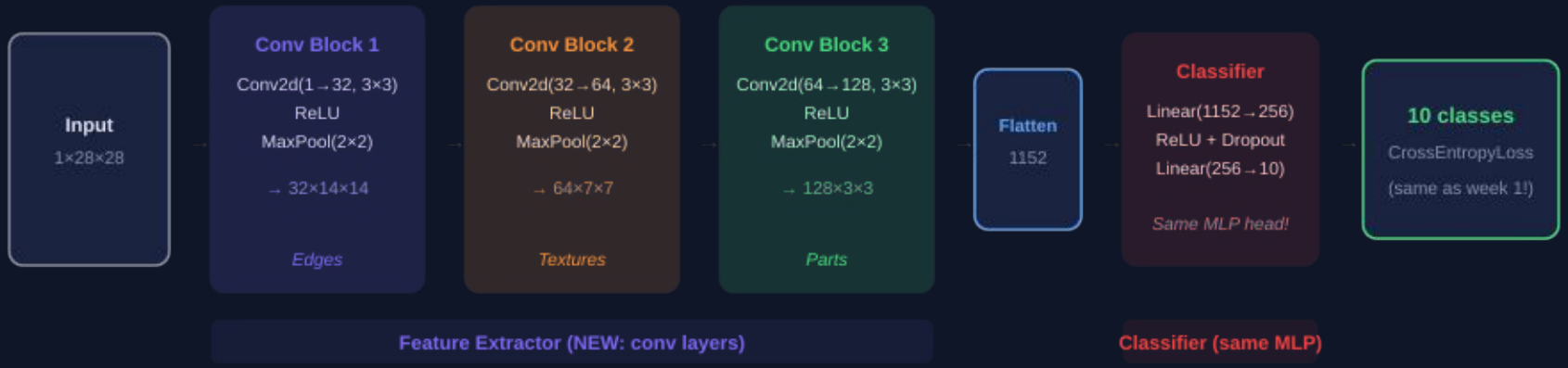
`nn.MaxPool2d(kernel_size=2, stride=2)` — halves H and W each time.

Fashion-MNIST size walkthrough: 28×28 → Conv → 28×28 → Pool → **14×14** → Conv → Pool → **7×7** → Conv → Pool → **3×3**

Spatial dims shrink. Channel count grows. Information gets compressed into feature vectors.

Our CNN Architecture

Conv blocks extract features, then the same MLP classifier from week 1 makes the decision



The Conv Block pattern: `Conv2d → ReLU → MaxPool2d`

Stack these blocks, increase channels (1→32→64→128), then flatten into a classifier.

Same 5-step training loop. Same CrossEntropyLoss. Same Adam optimizer. Same model.train() / model.eval().

The only new code is nn.Conv2d and nn.MaxPool2d in __init__.

Today's Coding Session

~10 min

Conv Intuition

Apply edge/blur kernels to a real image, see what convolution actually does

~17 min

Build & Train

CNN on Fashion-MNIST, same 5-step loop, compare accuracy to week 1's MLP

~10 min

Visualize

See learned filters and feature maps — what did the CNN actually learn?

+ 10 min Q&A at the end

Questions?

Week 3: CNN Fundamentals

IIT Madras · Wadhwani School of AI